

ClinicalBridge: Bridging the Clinical Context Gap

Final Project Report

Project: ClinicalBridge: Bridging the Clinical Context Gap

Course: COP-3442 Prompt Engineering

University: Bahçeşehir University

Department: Artificial Intelligence Engineering Department

Professor: Binnur Kurt

Group Members: - Kenan Eliyan — 2286181 - Rama Tamimi — 2285460 - Joud Wardeh — 2282493

Educational prototype. All patient data in this project is fictional. ClinicalBridge is not a real clinical tool, it does not make diagnoses, and clinician review is always required.

Executive Summary

Remote Patient Monitoring (RPM) devices generate alerts — "blood pressure 182/104", "glucose 312" — that reach a care team as bare numbers with no story attached. To act safely, a clinician must manually reconstruct the patient's context from three disconnected systems: the Electronic Health Record (EHR), the RPM stream, and the patient's own account (anamnesis). This manual reconstruction is slow, and important signals — a missed refill, a quietly worsening trend, a contradiction between systems — are easy to miss. This is the **Clinical Context Gap**.

ClinicalBridge is a proof-of-concept **multi-agent decision-support prototype** that closes that gap. A four-agent pipeline — **Alert Triage**, **EHR Retrieval**, **Anamnesis**, and **Synthesis**, coordinated by an **Orchestrator** — converts an RPM alert into a single **Clinical Context Brief** in which every clinical claim is traceable to its source, uncertainty and missing data are surfaced, conflicts are flagged rather than hidden, and no diagnosis is ever made.

The project is implemented as a **no-API prototype**: each agent's output is produced manually in the Claude application using engineered system prompts, stored as JSON, and replayed through real orchestration, schema-validation, and evaluation code. Across five hand-built scenarios the system achieves **0.0 hallucination rate** and **1.0 source traceability** with all five passing the safety checklist. Crucially, the report does **not** read these numbers as evidence of clinical readiness: the scores are high because the data is simulated and controlled, and a dedicated evaluation report documents the limits and three adversarial failure-mode tests. The genuine contribution is the **prompt-engineering methodology** — role scoping, cite-or-omit grounding, negative-space prompting, and a verify-before-emit self-check — that makes a multi-agent LLM system behave safely under hard constraints.

Problem Statement

A modern remote care team receives data from three silos that rarely interoperate:

- 1. **RPM** — live device readings (blood pressure, glucose, heart rate, SpO2, weight) plus the thresholds and baselines that turn a reading into an *alert*.
- 2. **EHR** — the formal record: diagnoses, medications, labs, allergies, and visit notes.
- 3. **Anamnesis** — the patient's own account: symptoms, history, medication adherence, lifestyle, family history, and concerns.

The same number means very different things in different contexts. A glucose of 312 mg/dL in a newly-diagnosed diabetic with no baseline is a different problem from the same value in a patient who just stopped their oral medication. **The number is constant; the context is everything.** Today, assembling that context is manual, repetitive, and error-prone, and the highest-value signals are precisely the ones that span systems (e.g., an EHR refill date that only becomes meaningful next to a patient-reported lapse).

A naive "just ask an LLM to summarize the chart" approach fails on the dimensions that matter in healthcare: it hallucinates, it states conclusions as fact, it silently resolves contradictions, and it cannot be audited. The problem this project addresses is therefore not only *assembling context* but doing so **safely, traceably, and without diagnosing**.

Project Objective and Scope

Objective. Build a multi-agent prototype that ingests one RPM alert, gathers the relevant EHR and anamnesis context, and produces a structured, fully-traceable Clinical Context Brief that supports — never replaces — clinician judgment.

In scope - Summarizing and citing existing (fictional) data. - Triaging alerts for *prioritization*. - Surfacing missing data and conflicts. - A measurable evaluation framework and a documented prompt-iteration history.

Explicitly out of scope (safety guardrails)

The system will...	The system will NOT...
Summarize and cite existing data	Diagnose or rank diagnoses
Flag urgency for prioritization	Decide treatment or write orders
Surface missing data and conflicts	Fill gaps with assumptions
Use only fictional data	Touch real patients / real PHI
Keep a human in the loop	Auto-act on any patient

System Architecture

The system is a sequential pipeline with one logically-parallel stage, coordinated by an orchestrator. Each arrow is a **typed JSON message** validated against a JSON Schema before the next stage runs.



- **Triage first** because it sets urgency *and* the retrieval plan, letting the two retrieval agents fetch targeted context rather than dumping the whole chart.
- **EHR + Anamnesis are independent** and read different sources; they are logically parallel.
- **Synthesis last** because it needs all three structured outputs to reconcile conflicts and build the traceability index.

Orchestrator responsibilities (implemented in `pipeline.py`): load data, sequence the agents, validate each hand-off against its schema, pass `retrieval_plan` forward, run guard rails (patient-ID consistency, non-empty disclaimer, non-empty traceability index, non-empty missing-data section), and assemble the final brief. A deliberate design choice is the **single seam to a real API**: in no-API mode `run_agent()` reads a stored output; swapping in a live model is a one-function change while orchestration, schemas, and evaluation stay identical.

The seven JSON Schemas in `schemas/` define the data contracts: three inputs (`rpm_alert`, `ehr_record`, `anamnesis_record`), three intermediate agent outputs (`triage_output`, `ehr_retrieval_output`, `anamnesis_output`), and the final `clinical_context_brief`.

Diagrams

Rendered architecture diagrams — high-level system, end-to-end data flow, the API-ready provider design, the prompt-iteration flow, and the evaluation pipeline — are available in `docs/09_architecture_diagrams.md`.

EHR retrieval backend: a genuine LangChain RAG pipeline

The EHR Retrieval Agent is grounded by a **real LangChain Retrieval-Augmented-Generation pipeline** implemented in `langchain_pipeline/`. It uses **local embeddings and a real vector store — with no external API keys** — so the project can honestly claim a genuine RAG backend while staying fully reproducible offline. The pipeline follows the standard load → split → embed → index → retrieve chain:

1. **LangChain Documents** (`ehr_loader.py`). Every EHR item in `data/patients.json` (diagnosis, medication, lab, allergy, visit note) is converted into a real `langchain_core.documents.Document`. The `page_content` holds the clinical text; the `metadata` carries `patient_id`, `source_id`, `source_type`, and `scenario_id` — so the same `source_id` citation handle the rest of the system relies on travels with every chunk.

2. **Text splitter** (`rag_pipeline.py`). A `RecursiveCharacterTextSplitter` (from `langchain_text_splitters`, `chunk_size=512`, `chunk_overlap=64`) chunks longer notes while preserving each parent's metadata, so a split note's chunks all retain their `source_id`.
3. **Local embeddings** (`vector_store.py`). Chunks are embedded with `sentence-transformers/all-MiniLM-L6-v2` via `langchain-huggingface` (`HuggingFaceEmbeddings`). The model runs locally; weights are cached on first use. There is **no OpenAI, no Anthropic, and no API key**. If `sentence-transformers` is unavailable, the pipeline falls back to a documented deterministic `FakeEmbeddings`, so it still runs fully offline.
4. **FAISS vector store** (`vector_store.py`). The embedded chunks are indexed in a **FAISS** store (`langchain_community.vectorstores.FAISS`). The corpus is tiny (71 EHR Documents across the 10 fictional patients), so the index is rebuilt in memory on startup; FAISS `save_local` / `load_local` persistence helpers are provided for completeness. If `faiss-cpu` is unavailable, a genuine `LangChain InMemoryVectorStore` is used instead.
5. **Retriever** (`retriever.py`). A configurable top-k retriever (default `k=5`) returns the most relevant EHR chunks for the clinical question, with FAISS similarity scores and a per-patient metadata filter so an alert only ever retrieves its own patient's chart.
6. **Integration with the EHR Retrieval Agent**. The orchestrator builds the clinical question from the **Alert Triage Agent's** `retrieval_plan.ehr_focus`, runs the `LangChain` retriever, and attaches the retrieved `Document` objects to the EHR Retrieval Agent's context (`context["ehr_retrieval_documents"]`) as its grounding evidence. This is wired in through a `RetrievalBackend` abstraction in `pipeline.py`: the default backend is the genuine `LangChain` pipeline, with the local TF-IDF retriever as the fallback. The four-agent orchestration, prompts, schemas, guard rails, and the 8-section brief are otherwise **unchanged** — `LangChain` replaces only the retrieval backend.

`python main.py --langchain-rag-demo N` shows the whole pipeline for one scenario: the configuration (embedding model, vector store, document/chunk counts), the clinical question, the retrieved `LangChain` Documents with metadata and source IDs, the FAISS similarity scores, and **Precision@5 / Recall@5** against the scenario's gold relevant facts. The measured retrieval quality is reported in `evaluation_report.md`.

Local TF-IDF fallback retriever

Before the `LangChain` backend was added, the project shipped a dependency-free retriever (`rag_retrieval.py`) and it is **retained as the documented fallback** so the project never breaks if the `LangChain` stack is not installed. It chunks each patient's EHR into one text chunk per item (each carrying its `source_id`), builds a **pure-Python TF-IDF** index, and ranks chunks against the clinical question by cosine similarity. `python main.py --rag-demo N` prints the question, the top-k retrieved chunks with scores and source ids, and the overlap with the gold relevant facts. It needs no embedding model, no vector database, and no network — the honest no-dependency baseline the genuine `LangChain` pipeline is compared against in the evaluation report.

Session memory and tools

Two further concepts are demonstrated minimally and honestly:

- **Session memory** (`session_memory.py` → `memory/session_memory.json`): a session-level audit log that records, per run, the scenario id, patient id, triage urgency, retrieved EHR source ids, and the final brief id. It is **audit memory, not long-term clinical memory** — it stores only ids and the triage label, no clinical narrative and no real patient data.
- **Tools** (`tools.py`): four small, named, documented tool wrappers an agent or the orchestrator could call — schema validation, EHR retrieval, patient-consistency check, and traceability check — each wrapping existing project logic rather than adding new behavior.

These additions are intentionally lightweight; they do **not** claim to implement full LangChain agents, a production vector database, or live LLM tool-calling.

Urgency taxonomy and critical-alert escalation

The official brief defines four urgency levels — **Critical / Urgent / Routine / Informational**. The project's internal schema enum predates that and uses `emergent / urgent / routine / monitor`. To align without invalidating the stored outputs, gold standards, and evaluation, the internal enum is **left unchanged** and a display mapping is applied (`pipeline.official_urgency()`); the rendered brief's Risk Assessment shows the official label:

Internal (schema)	Official (brief)
<code>emergent</code>	Critical
<code>urgent</code>	Urgent
<code>routine</code>	Routine
<code>monitor</code>	Informational

Critical-alert escalation. The brief requires that critical alerts escalate immediately. The orchestrator detects a Critical (`emergent`) triage result right after the triage step (`pipeline.is_critical()` / `escalation_notice()`) and surfaces an immediate escalation banner at the top of the brief: *"IMMEDIATE ESCALATION (Critical): requires immediate human clinician review now..."* — with **no treatment advice and no diagnosis**. None of the five shipped scenarios is Critical (they are Urgent/Routine), so existing behavior is unchanged; the rule is demonstrated on a synthetic triage by `python main.py` (the "Critical-Alert Escalation Logic" check) and documented in `ADVERSARIAL_TESTS.md`.

Dataset Design

All data is **synthetic**. The dataset (`data/patients.json`) contains **10 fictional patients**, each with a full EHR (diagnoses, medications with refill dates, labs, allergies, visit notes), an RPM profile (baselines, thresholds, recent readings, alerts), and an anamnesis record. A consistent **source-id convention** (`EHR.med.001` , `RPM.reading.003` , `ANAMNESIS.adherence.1` , ...) makes traceability machine-checkable.

Five patients anchor five scenarios, each engineered to stress a different failure mode:

Scenario	Patient	Stress test
1. Missed Medication	P001 (HTN/T2DM)	Connect an EHR refill gap to a patient-reported lapse
2. False Alarm	P002 (healthy cyclist)	De-escalate safely without dismissing a real reading
3. Silent Deterioration	P003 (heart failure)	Reason over a multi-day trend; beat the device's "medium" severity
4. Incomplete Record	P004 (new diabetic)	Anamnesis not collected — flag it, do not fabricate
5. Conflicting Data	P005 (T2DM, insulin)	EHR med list vs. patient report — surface, don't resolve

Each scenario file in `scenarios/` is self-contained: the RPM alert, the full EHR and anamnesis context, a **gold-standard brief**, and a `gold_eval` block (expected triage, the relevant source-id sets, and a synthesis checklist) used by the evaluation harness. The remaining five patients (COPD, post-MI, atrial fibrillation, CKD, asthma/prediabetes) provide realism and could seed further scenarios.

Agent Design

Each agent is intentionally narrow — a small job is easier to prompt, test, and keep safe — and every agent emits schema-valid JSON bound by a shared **cite-or-omit** rule.

- **Alert Triage Agent.** Input: the alert + RPM baselines/thresholds. Output: `urgency_level` (prioritization, *not* a diagnosis), `alert_validity` (`likely_valid` / `possible_artifact` / `insufficient_data`), and a `retrieval_plan` directing the next two agents. Key safety rule: when in doubt, triage up; an artifact is never simply dismissed.
- **EHR Retrieval Agent.** Input: the full EHR + alert + retrieval plan. Output: only the *relevant* EHR items, each copied **verbatim** with a source id and a relevance reason, plus a required `data_gaps` array (relevant data that *should* exist but is absent/stale). This is a relevance-and-grounding task, not a summarize-everything task.
- **Anamnesis Agent.** Input: the anamnesis record + alert + retrieval plan. Output: relevant patient-reported items across five domains, each tagged `reported_by: patient`, plus an `extraction_completeness` map distinguishing *absent* from *not collected*. Symptoms stay symptoms; they are never converted into diagnoses.
- **Synthesis Agent.** Input: the three structured outputs. Output: the Clinical Context Brief — contextual findings, *factors to consider* (never diagnoses), explicit `data_conflicts`, a required missing-data/uncertainty section, clinician *considerations* (not orders), an urgency assessment (keeps the higher concern), a safety disclaimer, and a traceability index mapping every cited id to EHR/RPM/ANAMNESIS.

Full specifications are in `docs/02_agent_design.md`; the production system prompts are in `prompts/<agent>/v3.md`.

Prompt Engineering Process

The hard part of this project is not the ~250 lines of Python — it is making four LLM agents stay in their lane, refuse to hallucinate, cite everything, express calibrated uncertainty, and hand structured JSON to one another. The following techniques were applied and are catalogued in

[docs/08_prompt_portfolio.md](#):

- **Role / persona prompting** to scope behavior and fix a non-diagnostic stance.
- **Rubric-in-prompt** (the urgency ladder) for consistent, reproducible judgments.
- **Cite-or-omit grounding** — every clinical claim carries a real source id, or it is not written.
- **Verbatim-copy** instruction to block dose/lab paraphrase drift in retrieval.
- **Source-typing** (`reported_by`) to prevent over-trusting self-report.
- **Negative-space prompting** — an explicit "what relevant data is *missing*?" requirement.
- **Few-shot exemplars** (the gold briefs) to lock tone and citation style.
- **Verify-before-emit** self-check in the synthesis prompt: re-walk every claim for a resolvable citation and scan for diagnosis language before output.
- **Constrained framing vocabulary** — "factors to consider", "patient-reported", "not a diagnosis".

Each agent was iterated through three versions (v1 naive → v2 structured & safe → v3 robust), kept in the repo as evidence of method. The detailed before/after analysis is in

[reports/prompt_engineering_portfolio.md](#).

Prompt Iteration Summary

The most consequential iteration was the **Synthesis Agent**, where the failures are exactly those the project exists to prevent:

Agent	v1 failure (observed)	Fix introduced	Result
Triage	Prose output that <i>named a likely disease</i>	JSON contract + urgency rubric + "no diagnosis" + "triage up"	Parseable; overt diagnosis removed
Triage (v2→v3)	Dismissed an implausible reading; anchored to device severity	<code>possible_artifact / insufficient_data ; ignore severity_raw</code>	Correctly escalates Scenario 3
EHR	Paraphrased a dose; no sources; silent gaps	source ids + verbatim rule + required <code>data_gaps</code>	Traceable; missing labs surfaced
Anamnesis	Hallucinated symptoms to fill an empty record	not-collected handling + "do not fabricate" + completeness map	Scenario 4 returns empty + flags; 0 fabrication
Synthesis	Wrote "patient is in acute heart failure; start IV diuresis" (diagnosis + order)	"factors to consider" framing + banned phrasing + cite-or-omit	Safety FAIL → PASS
Synthesis (v2→v3)	Silently chose EHR over patient in a conflict; dropped disclaimer	required <code>data_conflicts</code> + verify-before-emit + required disclaimer/index	Conflict surfaced; hallucination 0.0

Evaluation Framework

The system is graded with a measurable rubric applied per scenario, comparing output to the gold standard. Seven metrics are defined in `docs/04_evaluation_framework.md` and implemented in `evaluation/metrics.py`:

1. **Triage accuracy** — urgency + validity vs. gold (auto).
2. **Retrieval relevance** — precision/recall/F1 over the relevant source-id set (auto).
3. **Anamnesis completeness** — recall of gold ids + domain coverage (auto).
4. **Synthesis accuracy** — per-scenario gold checklist (human).
5. **Hallucination rate** — fraction of claims with a missing/unresolvable citation (auto). *Target 0.*
6. **Source traceability** — fraction of claims with a valid, resolvable citation (auto). *Target 1.*
7. **Safety compliance** — binary checklist; automated pre-checks plus human confirmation.

A supporting metric, **index coverage**, verifies that every cited id also appears in the brief's `traceability_index`. Metrics 1–3, 5, 6 are mechanizable; 4 and 7 require human judgment — a split that is itself a finding about what can and cannot be automated in prompt-quality assessment.

Evaluation Results

Running `python main.py` (or the notebook) over all five scenarios produces:

Scenario	Triage	Retr. F1	Anam.	Halluc. ↓	Trace. ↑	Idx cov.	Safety
1 Missed Medication	1.0	1.0	1.0	0.0	1.0	1.0	PASS
2 False Alarm	1.0	1.0	1.0	0.0	1.0	1.0	PASS
3 Silent Deterioration	1.0	1.0	1.0	0.0	1.0	1.0	PASS
4 Incomplete Record	1.0	1.0	1.0	0.0	1.0	1.0	PASS
5 Conflicting Data	1.0	1.0	1.0	0.0	1.0	1.0	PASS
Mean	1.0	1.0	1.0	0.0	1.0	1.0	PASS

These numbers must be read carefully. They show the system *can* behave correctly on well-formed, curated cases and that the v3 prompts achieve their design goals — but the data is **simulated and controlled**, the gold standard was authored by the same person who built the prompts, and only five happy-path scenarios plus three adversarial cases were tested. High scores here are necessary, not sufficient, evidence. The honest interpretation, the three adversarial failure-mode tests (missing RPM value, conflicting patient ID, vague symptom), and the limitations are detailed in [reports/evaluation_report.md](#) and [ADVERSARIAL_TESTS.md](#). The adversarial tests confirm the system **refuses to produce a brief** on malformed input rather than fabricating, which is the more important safety property than any accuracy score.

Ethical Considerations and Limitations

- **Not a medical device and not clinically validated.** ClinicalBridge has never been tested on real patients or real data, has no regulatory clearance, and makes no validated clinical claim. It must not be used for care.
- **Synthetic data only.** Results do not generalize to messy real-world EHR/RPM data, which contains noise, free-text, missingness, and errors far beyond these fixtures.
- **Automation bias.** Even a "support" tool can anchor a clinician. The brief deliberately leads with uncertainty, frames factors as hypotheses, and ends in *considerations* (not orders) to mitigate this — but the risk cannot be eliminated by design alone.
- **Evaluation is self-referential.** Gold standards were authored alongside the prompts; independent evaluators and held-out cases would be required for credible measurement.
- **No diagnosis by construction.** The system is constrained to "factors to consider"; this is a safety feature, but it also means the tool cannot and should not be relied upon for clinical conclusions.
- **What real use would require.** Clinical validation studies, regulatory review, a bias/fairness audit across demographics, robustness testing on real data, and a clinician-in-the-loop interface with full audit logging.

Bias Awareness

Even on fictional data, the design choices made here can encode bias, and a responsible capstone must name that explicitly:

- **The simulated cohort is small.** Ten fictional patients cannot represent population diversity; any apparent "performance" is over a handful of hand-authored cases.
 - **The synthetic data may not represent different ages, genders, cultures, languages, or socioeconomic contexts.** The patients are English-speaking with Western-style names, common chronic conditions, and tidy records; real populations are far more varied, and the system has never been tested against that variety.
 - **The prompts may encode assumptions** — about access to medication and transport (e.g., the "couldn't get a ride to the pharmacy" lapse), diet, adherence behavior, and communication style (clear, fluent self-report). Patients who communicate differently, face different barriers, or whose records are messier could be served less well, and those gaps would not show up in this evaluation.
 - **The gold standards reflect the authors' own assumptions** about what is "relevant" or "urgent," which is itself a potential source of bias in the metrics.
 - **Real deployment would require a bias/fairness review and diverse clinical validation** — testing across demographic groups, languages, and care settings, with independent clinical reviewers, before the system could be considered equitable or safe.
-

Reflection and Lessons Learned

What worked well. Decomposing one hard task into four narrow agents made every step easier to prompt, test, and keep safe — a small, well-scoped job is far easier to constrain than a single "read the chart and explain everything" prompt. The orchestrator + schema contracts meant the agents could be developed and validated independently, and the no-API design made the whole thing reproducible.

What failed in early prompt versions. The v1 prompts exposed exactly the dangers the project exists to prevent: the Synthesis agent wrote a diagnosis *and* a treatment order; the Anamnesis agent **hallucinated symptoms** to fill an empty record; the EHR agent paraphrased a medication dose and cited nothing. These were not model failures so much as *prompt* failures — the instructions left room for unsafe behavior, and the model took it.

Why structured JSON outputs helped. Forcing every agent to emit schema-valid JSON did more than make parsing easy. It turned vague prose into discrete, checkable fields (`data_gaps` , `extraction_completeness` , `data_conflicts`), which both *forced* the model to consider absence and conflict and *let* the evaluation mechanically verify them. A free-text summary cannot be audited; a typed object can.

Why source traceability was important. Requiring a real source id on every clinical claim ("cite-or-omit") is the single most effective anti-hallucination mechanism in the project. It makes fabrication structurally visible: an uncited or unresolvable claim is caught by the evaluation (hallucination rate) instead of slipping through as plausible prose. Traceability is what lets a clinician trust — and check — each line of the brief.

Why safety guardrails were necessary. The "factors to consider" framing, the banned-diagnosis language, the required disclaimer, the patient-ID consistency check, and the verify-before-emit step are not decoration — each one was added in response to a concrete failure (a diagnosis, an order, a

fabricated symptom, a silently-resolved conflict). Safety here is an accumulation of specific guardrails, not a single rule.

Why no-API / manual mode made the prototype reproducible. Generating each agent's output by hand in Claude and storing it as JSON means the demo, the brief, and the evaluation scores are identical on every run and on every machine — ideal for grading and recording — while keeping the focus on the graded skill (prompt engineering) rather than API plumbing.

How the API-ready provider design could support future work. Because the orchestrator calls a *provider* rather than a model directly, a live Claude connection can be added later by implementing one method (`ClaudeAPIProvider.run_agent`) with no change to the schemas, guard rails, or evaluation. The same test harness that scores stored outputs would then score live outputs.

Why high evaluation scores must not be read as clinical validation. The perfect scores reflect **simulated, controlled, self-authored** data and a tiny test set — they show the prompts *can* behave correctly and safely on clean cases, not that the system works on real patients. The honest headline is the **safe-failure** behavior on adversarial input, not the 1.0s. ClinicalBridge is an educational prototype on fictional data; it is not clinically validated and requires clinician review.

Future Work

- **Live Claude API integration** via the existing `ClaudeAPIProvider` seam, with multi-sample runs to measure real run-to-run variance.
- **A larger, messier simulated dataset** with noise, free-text, missing units, and contradictory timestamps to stress the agents beyond clean fixtures.
- **Scaling the LangChain retrieval stage.** The genuine LangChain + FAISS + local-embedding backend is now in place (`langchain_pipeline/`); the next step is testing it on charts with hundreds of items — adding metadata pre-filtering and a re-ranking stage — where precision, not recall, becomes the hard problem.
- **More adversarial scenarios** (unit mismatches, duplicate records, stale data, conflicting labs, multilingual input) to broaden the safe-failure coverage.
- **A clinician review study** with independent annotators authoring gold standards and scoring synthesis/safety blind, to remove the current self-referential bias.
- **Latency / performance testing** under realistic load once live API calls are introduced.
- **An optional UI or dashboard** for clinician-in-the-loop review with audit logging (presentation layer only; out of scope for this prototype).

Conclusion

ClinicalBridge demonstrates that a multi-agent LLM system can assemble fragmented clinical context into a single, fully-traceable, safety-constrained brief without diagnosing and without hallucinating on the tested cases. The engineering contribution is the **prompt methodology** — narrow role scoping, cite-or-omit grounding, negative-space prompting, source typing, and verify-before-emit — documented through a v1→v3 iteration history and measured with a seven-metric rubric. The decisive results are not the perfect scores on curated data but the system's **safe-failure behavior**: it refuses

malformed alerts, halts on patient-identity mismatch, surfaces conflicts instead of resolving them, and flags vague or missing data rather than inventing specifics. With the explicit caveat that this is an educational prototype on fictional data requiring clinician review, ClinicalBridge is a working illustration of how prompt engineering — not model power alone — produces trustworthy behavior in a high-stakes domain.

References

The problem framing draws on the documented EHR/RPM/anamnesis failures noted in the brief: EHR problem-list incompleteness (Wright et al., 2015), monitor alarm fatigue and low actionable-alert rates (Cvach, 2012; Hravnak et al., 2018), and the structural disconnection of digital anamnesis (Bachmann & Windak, 2020). The methodology draws on established prompt-engineering techniques — chain-of-thought reasoning (Wei et al., 2022) and the reason-then-act (ReAct) pattern (Yao et al., 2023) — and on retrieval-augmented generation as described in the LangChain documentation; the cite-or-omit grounding rule is this project's response to the hallucination risk highlighted for LLMs in medicine (Singhal et al., 2023; Thirunavukarasu et al., 2023).

Clinical context gap and health informatics 1. Wright, A., et al. (2015). *Problem list completeness in electronic health records: A multi-site study and assessment of success factors*. International Journal of Medical Informatics. 2. Senbekov, M., et al. (2020). *The recent progress and applications of digital technologies in healthcare: A review*. International Journal of Environmental Research and Public Health. 3. Adler-Milstein, J., & Jha, A. K. (2017). *HITECH Act drove large gains in hospital EHR adoption*. Health Affairs. 4. Hravnak, M., et al. (2018). *Cardiorespiratory instability before and after implementing an integrated monitoring system*. Critical Care Medicine.

Remote patient monitoring 5. Cvach, M. (2012). *Monitor alarm fatigue: An integrative review*. Biomedical Instrumentation & Technology. 6. Vegesna, A., et al. (2017). *Remote patient monitoring via non-invasive digital technologies: A systematic review*. Telemedicine and e-Health. 7. Noah, B., et al. (2018). *Impact of remote patient monitoring on clinical outcomes: An updated meta-analysis of randomized controlled trials*. NPJ Digital Medicine.

Anamnesis and patient-reported data 8. Bates, D. W., et al. (2014). *Big data in health care: Using analytics to identify and manage high-risk and high-cost patients*. Health Affairs. 9. Bachmann, M., & Windak, A. (2020). *Digital anamnesis in primary care: Potential and limitations*. MDPI Healthcare.

LLM applications in healthcare 10. Singhal, K., et al. (2023). *Large language models encode clinical knowledge*. Nature. 11. Nori, H., et al. (2023). *Capabilities of GPT-4 on medical competence examinations*. arXiv. 12. Thirunavukarasu, A. J., et al. (2023). *Large language models in medicine*. Nature Medicine.

Prompt engineering and multi-agent systems 13. Wei, J., et al. (2022). *Chain-of-thought prompting elicits reasoning in large language models*. NeurIPS. 14. Yao, S., et al. (2023). *ReAct: Synergizing reasoning and acting in language models*. ICLR. 15. Park, J. S., et al. (2023). *Generative agents: Interactive simulacra of human behavior*. UIST. 16. LangChain Documentation. *Retrieval-Augmented Generation and Agent Frameworks*.

Tools and course materials 17. Anthropic. *Claude* (model used to generate agent outputs manually in no-API mode). 18. JSON Schema (draft-07) specification — used for all data contracts. <https://json-schema.org/> 19. Course materials, COP-3442 Prompt Engineering (Bahçeşehir University, AI Engineering Dept.).

Reminder: all clinical content in this report is illustrative and based on fictional data created for coursework. Nothing here is medical advice.

ClinicalBridge - educational prototype. All data is fictional. Not a real clinical tool. No diagnoses. Clinician review always required.